Images

Use the Dynamic Imaging Service (DIS) and the `<DynamicImage>` component to deliver optimized, responsive images in your storefront. DIS formats the correct size for each image application and eliminates the need of uploading multiple images with different sizes.

Images are typically the largest contributor to page weight. Unoptimized images degrade Core Web Vitals by increasing Largest Contentful Paint (LCP) and slowing Time to Interactive (TTI). Storefront Next provides built-in DIS integration that handles format conversion, server-side resizing, and responsive source generation automatically.

Dynamic Imaging Service (DIS)

Salesforce B2C Commerce's Dynamic Imaging Service (DIS) is an image transformation service that optimizes images on-the-fly. Instead of storing pre-generated image variants, DIS transforms images at request time based on URL parameters. CDNs in front of DIS cache the transformed results at the edge.

DIS provides:

- Format conversion: Serves modern formats like WebP (25–35% smaller than JPEG/PNG) with automatic fallback.
- Server-side resizing: Sends each device exactly the pixels it needs.
- Quality control: Balances visual fidelity against file size.

DIS URL Anatomy

Storefront Next rewrites static B2C Commerce image URLs into DIS URLs with transformation parameters:

```
Original (static asset):
https://example.dx.commercecloud.salesforce.com/on/dem

DIS URL:
https://edge.dis.commercecloud.salesforce.com/dw/image
        └──────────────── DIS Host ────────────────┘
```

Parameters used by `<DynamicImage>`:

		width in pixels. When used alone, aspect ratio is preserved.	
`sh`	`scaleHeight`	Scale height. When combined with `sw`, scales to exact dimensions (constraining aspect ratio).	`sh=480`
`q`	`quality`	Quality: 1 to 100. Controls compression level.	`q=70`
`sfrm`	–	Source format. Tells DIS the original format for transcoding.	`sfrm=jpg`

The file extension in the URL path determines the **output** format (for example, `.webp`), while `sfrm` records the original format.

Additional DIS parameters not used by `<DynamicImage>` (but available for custom URL construction):

Parameter	Full Name	Description
`sm`	`scaleMode`	Controls scaling behavior: `fit` (default, fits within `sw` × `sh` preserving aspect ratio), `cut` (fills `sw` × `sh` and crops overflow)
`cx`, `cy`, `cw`, `ch`	`cropX`, `cropY`, `cropWidth`, `cropHeight`	Pixel-precise crop region. All four paramters must be specified together.
`bgcolor`	–	Background color for transparent areas (6-digit hex, for example `bgcolor=FFFFFF`)
`strip`	–	Remove image metadata (for example, EXIF)

Configuration

Configure DIS behavior in `config.server.ts` under the `images` key:

```
images: {
    quality: 70,            // Default DIS quality (1-
```

Override these values per environment with environment variables:

```
PUBLIC__app__images__quality=80
PUBLIC__app__images__enableDis=false
PUBLIC__app__images__host=https://edge.dis.commerceclo
```

When `enableDis` is `false`, the image system falls back to serving static assets directly. Format conversion, server-side resizing, and `<source>` generation are all skipped.

Image Filtering on Product Listing Pages

Search responses (`fetchSearchProducts`) include an `imageGroups` array on every hit. By default, B2 Commerce API (SCAPI) returns every imageGroup for every variant, which on variant-heavy catalogs can be the dominant contributor to PLP payload size–most of those images are never rendered.

The template restricts the response via SCAPI's `imgTypes` query parameter using `config.server.ts` :

```
search: {
  products: {
    images: {
      tile: 'medium',
      swatch: 'swatch',
    },
  },
}
```

Each role names the viewType a specific consumer reads: `tile` for the product tile hero, and `swatch` for the color thumbnails. The search filter derives its `imgTypes` query parameter as the union of these values (deduplicated and joined with `,`), so adding a new role automatically widens the filter. Setting a role to `undefined` opts that role out. Setting all roles to `undefined` , or providing an empty `images: {}` , disables filtering entirely and returns the full payload. `imgTypes` requires `expand=images` and `allImages=true` –both are set by `fetchSearchProducts` .

Keeping Role-Named Values Aligned with Consumers

If you customize the product tile to read a different viewType (for example, switch the hero from `medium` to `large`), you must update the matching role here. Otherwise, the tile will receive empty image arrays for the unrequested viewType. The built-in consumers that should eventually read from these declarations are:

`swatchViewType` to `"swatch"`)

The hardcoded strings in those consumers are tracked for a followup cleanup that derives them from these same role-named declarations, eliminating drift.

DynamicImage Component

`<DynamicImage>` is a responsive image component that generates an optimized `<picture>` element with DIS-powered `<source>` elements and responsive preloading via React 19's `preload()` API.

```
1 import { DynamicImage } from "@/components/dynamic-ima
```

Basic Usage

```
1 <DynamicImage src="https://example.com/image.jpg" alt=
```

This renders a `<picture>` element with `<source>` elements sized per breakpoint, each requesting a DIS-resized WebP variant with 1x and 2x srcSet descriptors.

The src Prop and Placeholder Syntax

The `src` prop accepts plain URLs or URLs with **placeholder syntax**: bracket-delimited segments that `DynamicImage` replaces with computed values.

```
1 // Plain URL. DynamicImage appends sw/sh/q params auto
2 <DynamicImage src="https://example.com/image.jpg" widt
3
4 // Placeholder syntax. {width} and {height} are replac
5 <DynamicImage src="https://example.com/image.jpg[?sw={
6
7 // Inline placeholder in path segment
8 <DynamicImage src="https://example.com/image[_{width}]
```

The bracket syntax `[...]` marks optional URL segments that are stripped when no dimensions are provided.

Responsive Widths

The `widths` prop controls how wide each `<source>` requests its image from DIS. It determines the `sw` parameter value and the `sizes` attribute in the generated markup. It accepts three formats:

Array of numbers (interpreted as px):

```
// Mixed units: vw for fluid layouts, px for fixed lay
<DynamicImage src={imageSrc} widths={["100vw", "50vw",
```

When using `vw` units, `DynamicImage` calculates the actual pixel width at each breakpoint to request the correct size from DIS.

Object with breakpoint keys (maps to Tailwind's default breakpoints):

```
<DynamicImage src={imageSrc} widths={{ base: 400, sm:

// With units
<DynamicImage src={imageSrc} widths={{ base: '100vw',
```

Breakpoint keys correspond to Tailwind's default theme: `base`, `sm`, `md`, `lg`, `xl`, `2xl`. Values are carried forward: `{ base: 400, lg: 800 }` produces `[400, 400, 400, 800]`.

Use fixed px widths when the image container has a predetermined size (for example, carousels). Use vw-based widths when the image scales with the viewport (for example, product grids, hero banners).

Server-Side Scaling with Heights

The `heights` prop enables DIS server-side scaling via the `sh` parameter. When provided alongside `widths`, it defines exact output dimensions, giving you precise aspect ratio control across responsive breakpoints.

```
// 4:3 aspect ratio maintained across all breakpoints
<DynamicImage
  src="https://example.com/image.jpg[?sw={width}&sh={h
  widths={[400, 800, 1200]}
  heights={[300, 600, 900]}
/>
```

Both values are multiplied by the DPR factor. At 2x, `widths={[400]}` and `heights={[300]}` generates srcSet entries for `sw=400&sh=300` (1x) and `sw=800&sh=600` (2x).

`heights` supports the same formats as `widths` (arrays, objects with breakpoint keys).

When `heights` is omitted, DIS preserves the original aspect ratio based on `sw` alone.

Loading Priority and Preloading

`DynamicImage` integrates with React 19's `preload()` to emit `<link rel="preload">` hints for high-priority images during server rendering:

```
// Auto priority (default). Determined by DynamicImage
<DynamicImage src={productImage} widths={[...]} />

// Explicitly low priority. Lazy-loaded, no preload hi
<DynamicImage src={belowFoldImage} widths={[...]} prio
```

When `priority` isn't set, the component checks the `DynamicImageProvider` context to determine whether the image should be treated as high priority. If no context is present, it defaults to `'auto'` priority with `loading="lazy"`.

DynamicImageProvider Context

The `DynamicImageProvider` is an optional React context that controls image priority and dimensions for nested `<DynamicImage>` components. It solves a practical problem: in deep component trees (for example, product grid → product tile → product image), determining whether an image is above-the-fold requires knowledge the image component itself doesn't have. The provider bridges that gap by separating the decision about importance from the rendering of individual images.

```
import DynamicImageProvider from "@/providers/dynamic-
```

How It Works

The provider exposes two interfaces: one for the outer container that sets up the context, and one for the nested consumers that interact with it.

Container interface (passed via `value` prop):

```
value: {
    sources?: Set<string>;
    widths?: DynamicImageDimensions;
    heights?: DynamicImageDimensions;
    addSource?: (src: string, sources: Set<string>) =>
    hasSource?: (src: string, sources: Set<string>) =>
}
```

The container receives the raw `Set<string>` alongside each `src`, giving it full control over the registration and lookup logic.

Consumer interface (returned by `useDynamicImageContext()`):

```
{
  addSource: (src: string) => boolean; // Register thi
  hasSource: (src: string) => boolean; // Is this imag
  widths: DynamicImageDimensions | undefined;
```

their dimensions. The `<DynamicImage>` component calls `hasSource(src)` internally: when it returns `true`, the image is promoted to `priority="high"` and `loading="eager"`.

This separation means the container owns all priority decisions while nested components remain generic and reusable.

Example: Product Grid with Critical and Non-Critical Images

Split product grid tiles into critical (above-the-fold) and non-critical (below-the-fold) batches:

```
const responsiveImageWidths = [
    '40vw', // base: 2 columns
    '25vw', // sm: 3 columns
    '18vw', // md: 4 columns
    '14vw', // lg: 4 columns with refinement panel
    '16vw', // xl
    '16vw', // 2xl
];

// Critical tiles: all images are high priority
const hasSource = useCallback(() => true, []);

<DynamicImageProvider value={{ hasSource, widths: resp
    {criticalProducts.map(product => <ProductTile ...
</DynamicImageProvider>

// Non-critical tiles: no hasSource → all images defau
<DynamicImageProvider value={{ widths: responsiveImage
    {nonCriticalProducts.map(product => <ProductTile .
</DynamicImageProvider>
```

The `ProductTile` component itself is identical in both batches. It doesn't know whether it's above or below the fold. The provider controls that from the outside.

Example: Capping High-Priority Images with a Single Provider

A single provider can use the shared `Set<string>` to cap how many images are promoted. This example treats only the first row of a four-column grid as high priority:

```
const addSource = useCallback((src, sources) => {
    if (sources.size < 4) { sources.add(src); return t
    return false;
}, []);
const hasSource = useCallback((src, sources) => source

<DynamicImageProvider value={{ addSource, hasSource, w
    {allProducts.map(product => <ProductTile ... />)}
</DynamicImageProvider>
```

Utility Functions

The `@/lib/images/dynamic-image` module exports utilities for working with DIS URLs outside the `<DynamicImage>` component. Use these when you need to transform image URLs programmatically, for example in content slots or rich text from SCAPI.

Performance Checklist

Follow these guidelines to get the best performance from your storefront images:

- Use modern image formats: Modern image formats like WebP give 25–35% smaller files than JPEG/PNG. A `fallbackFormat` config allows the definition of an automatic fallback format for browsers that don't support any of the `<source>` formats.
- Above the fold: Set `priority="high"` and `loading="eager"` on LCP-candidate images. This triggers React 19 SSR preloading.
- Below the fold: Use `loading="lazy"`. Omit priority or set `priority="low"`.
- Always set `widths` or `heights`: Without either, `<DynamicImage>` renders a plain `<img>` with no responsive sources. The browser downloads the full-size image regardless of viewport.
- Prefer vw for fluid layouts: Use vw-based widths (for example, `'50vw'`) when the image width scales with the viewport. Use px-based widths when the image has a fixed maximum size (for example, product detail at `'680px'`).
- Set width/height on non-DynamicImage `<img>` elements: Always include explicit `width` and `height` attributes on standard `<img>` elements to prevent Cumulative Layout Shift (CLS). `<DynamicImage>` handles this via its responsive `<picture>` and sizing attributes.
- Use `DynamicImageProvider` for grids: Wrap product grids in a provider to control priority centrally rather than passing props through every tile.
- Tune quality per use case: A quality of 70 is a good baseline. Hero banners or product zoom may benefit from higher values (80–85). Thumbnails and carousels can go lower (50–60). Override globally per-environment via `PUBLIC__app__images__quality`, or per-image by adding the `q` parameter to the src URL (for example, `src="https://example.com/image.jpg?q=85"`). A `q` parameter present in the src URL takes priority over the global config.

Alt Text Strategy

Providing meaningful alt text is essential for accessibility and SEO.

Fallback Order

Use this fallback order for product images.

1. SCAPI image alt (`image.alt`)
2. Product name (`productName` / `name`)
3. Localized generic fallback (for example, `t('common:productImageAlt')`)
4. Non-localized English fallback as a final safety net (for example, `'Product Image'`)

Use explicit `||` fallback chains in components to preserve this order.

Rules

- Always provide an `alt` attribute on rendered `<img>` elements.
- Use localized strings for generic fallback alt text, then a hardcoded English fallback as the last resort.
- Decorative images must set `alt=""` when the image is purely decorative and has no meaningful text equivalent.

See Also

- Dynamic Imaging Service (Official Salesforce DIS documentation)
- Static Assets (Serving static files from the public directory)
- UI Styling (Tailwind CSS and design tokens)